TECHNICAL PROJECT PROPOSAL

TrustChain-IDRS

A Hardware-Integrated, Blockchain-Backed Intelligent Intrusion Detection and Response System for Smart Homes

Prepared for internal project review by the development team and faculty guide
Department of Computer Science and Engineering

Document status: Draft for internal circulation. Not for public disclosure.

Table of Contents

1. Abstract
2. Field of the Work
3. Background and the Problem Being Solved
4. Related Work and Prior-Art Landscape (Preliminary)
5. Summary of the System
6. System Architecture
7. Detailed Module Description
8. Novelty Assessment
9. Candidate Technical Contributions
10. Design Rationale and Scope Decisions
11. Implementation Roadmap
12. Indicative Bill of Materials
13. Risks, Limitations & Open Questions
14. Next Steps

1. Abstract

TrustChain-IDRS is a layered intrusion-detection-and-response architecture for consumer smart-home systems that combines (a) device-bound cryptographic identity and signed telemetry, (b) a hybrid rule-based and machine-learning (Isolation Forest) detection engine with an adaptive, time-of-day-aware baseline, (c) a continuously updated per-device trust score that drives automated quarantine of misbehaving devices, (d) a hardware-actuated physical response layer (visual/audible alarm, lock-state control), and (e) a permissioned blockchain layer that stores cryptographic hashes and signed metadata of security-relevant events — not raw sensor data — to create a tamper-evident audit trail that can independently detect after-the-fact modification of local logs.

The system is implemented on commodity ESP32 hardware and is intended as a working proof-of-concept and final-year research project deliverable. This document sets out the system architecture, distinguishes the elements considered potentially novel from well-established building blocks, and presents a phased implementation plan for team execution.

2. Field of the Work

This work relates generally to Internet-of-Things (IoT) security, and more particularly to methods and systems for intrusion detection, device trust management, and tamper-evident security auditing in networked embedded/home-automation environments, using a combination of cryptographic device identity, behavioral anomaly detection, dynamic trust scoring, automated physical/electronic response, and distributed-ledger-based audit logging.

3. Background and the Problem Being Solved

Consumer smart-home deployments typically consist of low-cost microcontrollers (e.g., ESP32) reporting telemetry and accepting commands over local Wi-Fi, with detection logic — if present at all — running as simple threshold rules on a central hub. This creates three recurring weaknesses that the proposed system is designed to address:

- No verifiable device identity: telemetry is trusted purely because it arrives from a known IP/MAC address, which is trivial to spoof or replay.
- No tamper-evidence for the security log itself: if the machine running the IDS is compromised, an attacker who can edit application logs can erase evidence of their own intrusion, and the system has no independent way to detect that the log was altered.
- Static, one-size-fits-all detection thresholds: fixed rules (e.g., “request rate > 10/s = attack”) generate false positives at some times of day and miss slow/low attacks at others, because normal household behavior is not constant across the day.

NIST's IoT device cybersecurity guidance (NISTIR 8259 / SP 800-213 series) frames exactly these gaps as core “device cybersecurity capabilities” that manufacturers should provide: device identification, authorized configuration, audit-event generation with protected/verifiable audit data, and cybersecurity state awareness. This project treats that guidance as a design checklist rather than as something to cite decoratively.

4. Related Work and Prior-Art Landscape (Preliminary)

This section is a non-exhaustive orientation, not a substitute for a professional prior-art search (see Section 14). Each of the following building blocks is independently well documented in the literature and in existing products, and a patent examiner will treat them as known:

- Hash-chained / blockchain-anchored audit logs for tamper evidence — a long-standing and widely published technique (well beyond IoT) for detecting after-the-fact log modification.

- Decentralized identifiers (W3C DID) and public-key-based device identity for IoT authentication.
- Isolation Forest and other unsupervised ML methods for network/IoT anomaly detection — an active, heavily published research area.
- Device trust/reputation scoring in IoT and zero-trust network access literature.
- Secure boot and signed OTA firmware update mechanisms, already supported natively by the ESP-IDF secure-boot and flash-encryption features on ESP32-class chips.

None of these building blocks is, by itself, likely to be patentable at this point. What a patent claim would need to rest on — discussed in Sections 8–9 — is a specific, non-obvious way of combining and operationally linking them (for example, the exact mechanism by which a blockchain-integrity check result feeds back into the trust score, which in turn triggers physical actuation), rather than the presence of blockchain, ML, or DIDs individually.

5. Summary of the System

The proposed system integrates, in a single feedback loop, the following elements:

- Cryptographically identified devices that sign each telemetry/event message with a device-bound key, and include a monotonic timestamp and single-use nonce to prevent replay.
- A two-stage detection pipeline: lightweight rule checks and replay/signature verification at a security-gateway node, followed by rule + ML (Isolation Forest) scoring at a central engine, producing a continuous risk value rather than a binary alert.
- An adaptive baseline that models expected behavior per time-of-day / device, so the same raw event can be scored differently depending on learned normal household patterns.
- A per-device trust score that is decremented by specific event classes (failed auth, replay, abnormal traffic, blockchain-integrity mismatch, firmware-hash mismatch) and that automatically triggers quarantine — a defined, reversible state in which the gateway blocks outbound commands to/from that device — when trust falls below a threshold.
- A permissioned blockchain audit layer that stores only hashes, device IDs, risk scores, and state-transition metadata for security-relevant events (not raw sensor data), enabling independent, after-the-fact verification that the locally stored event log has not been altered.
- A physical response layer (visual/audible indicators and lock actuation on a dedicated ESP32 node) whose state is driven directly by the trust/risk pipeline, so a detected critical event produces an observable, hardware-level consequence, not just a dashboard alert.

The claimed value is in the closed loop between these parts — particularly the trust-score → quarantine → physical-actuation path and the blockchain-integrity-check feeding back into trust — rather than in any single component.

6. System Architecture

The system is organized into five layers, shown below. Only hashes and metadata cross into the blockchain layer; raw telemetry stays in a local database, which keeps on-chain storage small and avoids putting potentially sensitive household data (occupancy patterns, routines) into an immutable, widely-replicated ledger.

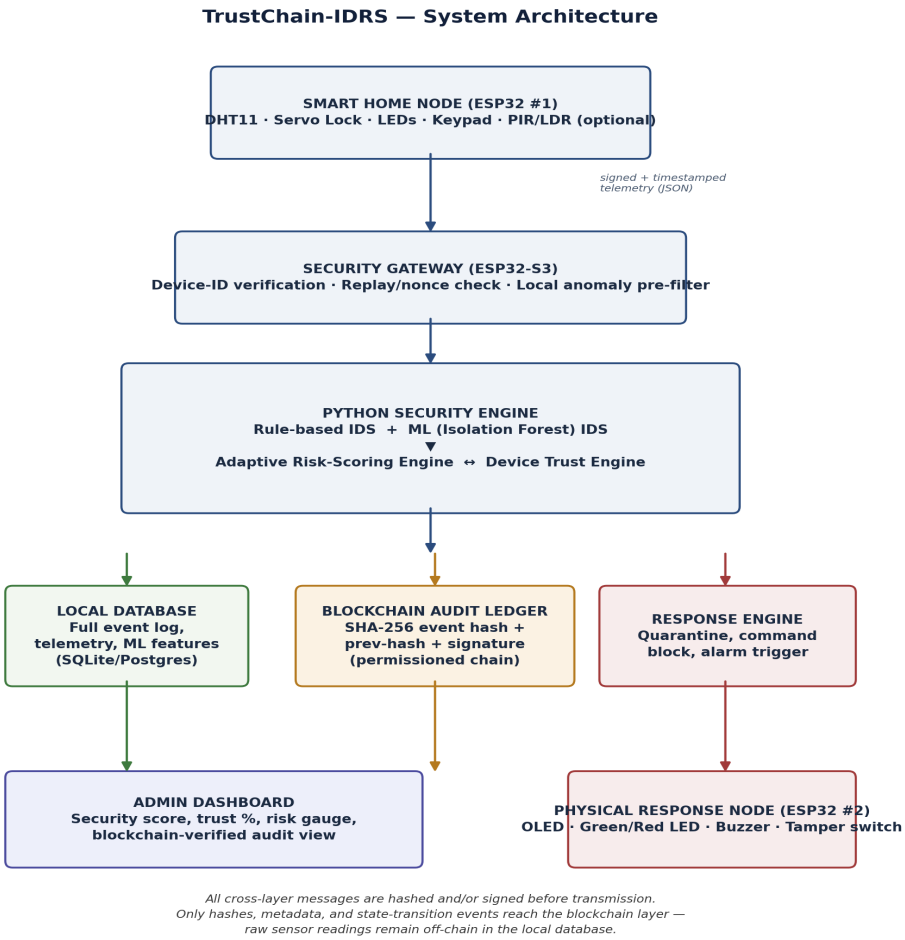


Figure 1. TrustChain-IDRS layered architecture and data flow.

At a message level, a representative signed telemetry payload from the smart-home node is:

```
{ "device_id": "did:trustchain:esp32:001", "timestamp": 1789140000, "nonce": "8FA32D",
"temperature": 31.7, "humidity": 70.1, "signature": "3045...af" }
```

and the corresponding audit record committed to the blockchain layer after a security event is only:

```
{ "event_id": "EVT-004821", "device_id": "did:trustchain:esp32:001", "event_type": "BRUTE_FORCE",
"risk_score": 0.94, "event_hash": "9e3a...a812", "prev_hash": "77b1...0e4f", "signature":
"3046...12", "trust_after": 42 }
```

7. Detailed Module Description

7.1 Module A — Smart Home Node

ESP32 running environmental sensing (DHT11), a servo-actuated lock, status LEDs, and a keypad for local authentication. This module already exists in the current prototype and requires the addition of a signing routine and nonce/timestamp fields to outgoing telemetry.

7.2 Module B — Security Gateway

A second, dedicated ESP32 (ESP32-S3 recommended for its hardware crypto acceleration) that verifies device signatures, rejects replayed nonces/stale timestamps, and performs a cheap first-pass anomaly check before forwarding to the Python engine. This is a new module relative to the original two-ESP32 design; see Section 10 for the reasoning.

7.3 Module C — Intelligent Detection Engine

Combines deterministic rules (rate limits, failed-PIN counters, known attack signatures) with an Isolation Forest model over engineered features (request rate, inter-arrival time, failed-auth count, sensor-value deviation). A per-hour, per-device baseline is maintained so the same raw feature value is scored against locally learned ‘normal,’ rather than one static global threshold.

7.4 Module D — Trust & Risk Engine

Maintains a 0–100 trust score per device, updated by defined deltas per event class, and computes a continuous risk value (weighted function of rule severity, ML anomaly score, and current trust) mapped to SAFE / SUSPICIOUS / HIGH RISK / CRITICAL bands that drive the response engine.

7.5 Module E — Blockchain Audit Layer

A permissioned ledger (see Section 10 for the recommendation to start with a lightweight hash-chain rather than a full Hyperledger Fabric deployment) that records device registration, firmware-version changes, security events, risk-band transitions, and quarantine/recovery actions as hash-anchored, signed entries. A background integrity checker periodically recomputes hashes of the local database and compares them against the chain to raise an AUDIT_TAMPERING alert on mismatch.

7.6 Module F — Automated Response

Encodes the quarantine mechanism: when trust drops below a configured threshold, the gateway blocks outbound commands to/from the affected device and the response node raises a physical alarm; recovery requires an explicit administrator action, which is itself logged and hashed.

7.7 Module G — Hardware Security Node

A dedicated ESP32 with OLED status display, red/green LED, buzzer, and an optional tamper switch that renders CRITICAL-risk states as an unambiguous physical signal, independent of whether the household dashboard is being watched.

8. Novelty Assessment

This section distinguishes established prior art from the elements of the proposed architecture considered potentially novel. This distinction is maintained throughout the document so that the team's engineering effort is directed toward the mechanisms most likely to constitute a genuine technical contribution.

8.1 Established prior art

- Use of a blockchain / hash-chain as a tamper-evident log — a long-standing and widely published technique, in and out of IoT.
- Decentralized identifiers / public-key-based device identity for IoT authentication.
- Isolation Forest and other unsupervised ML methods for network/IoT anomaly detection — an active, heavily published research area.
- Device trust/reputation scoring in IoT and zero-trust network access literature.
- Secure boot and signed OTA firmware update mechanisms, already supported natively by ESP-IDF secure-boot and flash-encryption features on ESP32-class hardware.
- Storing hashes rather than raw data on-chain — a standard, widely used anchoring pattern.

8.2 Candidate novel contributions

- A feedback mechanism in which the outcome of the blockchain integrity check (match/mismatch between a locally stored event and its on-chain hash) is fed as an input into the per-device trust-score calculation — ledger tampering itself lowers the trust of the implicated device and can trigger quarantine, rather than blockchain integrity being a separate, passive audit feature.
- The specific combination and ordering of signals (rule severity, ML anomaly score, adaptive time-of-day baseline deviation, trust score, ledger-integrity status) into a single bounded risk value that is used, in real time, to drive physical hardware actuation (lock/alarm), as distinct from using any one signal alone or using them only for offline reporting.
- A reversible, administrator-gated quarantine state machine that spans both the network layer (command blocking at the gateway) and the physical layer (lock/alarm state) and is itself an auditable, hash-anchored event.

These are preliminary technical observations. A formal patentability assessment would require a professional prior-art search, which is outside the scope of the present document.

9. Candidate Technical Contributions

The following statements describe the technical mechanisms considered most significant within the proposed architecture. They are presented in a structured, numbered format for clarity and internal review; they are not formatted as legal patent claims.

1. A method for securing a networked home-automation system, comprising: receiving, at a gateway, a telemetry or event message signed with a device-bound cryptographic key and accompanied by a timestamp and single-use nonce; verifying the signature and rejecting the message if the nonce has been previously observed or the timestamp is outside a permitted window; computing a risk score for the message from a combination of rule-based checks, an unsupervised machine-learning anomaly score, and a deviation from a time-of-day-adaptive behavioral baseline for the originating device; and updating a persistent trust value for the originating device based at least in part on the computed risk score.
2. The method of item 1, further comprising: committing a cryptographic hash of the event, together with the computed risk score and the resulting trust value, to a permissioned distributed ledger, while retaining the full event content in a separate local data store not replicated on the ledger.

3. The method of item 2, further comprising: periodically recomputing a hash of an event stored in the local data store; comparing the recomputed hash to the corresponding hash recorded on the distributed ledger; and, upon detecting a mismatch, decreasing the trust value of the associated device and generating a tampering-alert event, such that ledger-integrity verification is a direct input to device trust rather than a separate reporting function.
4. The method of item 1, further comprising: automatically placing the originating device into a quarantine state when its trust value falls below a configured threshold, wherein the quarantine state (i) causes the gateway to block outbound commands to and from the device, and (ii) causes a physically separate response node to actuate a human-perceptible alarm and/or a lock mechanism, and wherein exiting the quarantine state requires an explicit, separately logged administrator action.
5. A system configured to perform the method of any of items 1–4, comprising a plurality of microcontroller-based sensor/actuator nodes, a gateway node, a risk-scoring engine, and a permissioned distributed ledger.

10. Design Rationale and Scope Decisions

The following decisions define the scope of the system to be built, so that team effort is concentrated on the mechanisms identified in Section 8.2 as the system's principal technical contribution.

- Full W3C DID-ecosystem compliance, a dedicated secure-element chip (ATECC608-class), and Hyperledger Fabric are treated as future extensions rather than core deliverables. Device-bound key pairs generated and stored in ESP32 NVS/flash, together with a lightweight custom hash-chain (a linked list of SHA-256-hashed, signed blocks replicated across two or three nodes), demonstrate the same properties with substantially less implementation effort.
- Secure boot and signed OTA are treated as optional hardening rather than a core deliverable, since ESP-IDF already provides flash encryption and secure boot as built-in features; enabling and documenting them is a reasonable extension once the core system is complete.
- The blockchain layer is scoped to a single load-bearing function — the tamper-evidence and trust-feedback mechanism described in Section 8.2 — rather than being used as a general-purpose data store.
- The quarantine mechanism is defined as a state machine with an explicit, administrator-gated exit condition, so that automated response is demonstrably reversible rather than one-directional.
- Fixed global thresholds in the rule engine are replaced with a per-device, per-time-of-day adaptive baseline, addressing the false-positive/false-negative trade-off inherent in static thresholds.
- The system is organized into seven modules (A–G) with a defined dependency order, supporting the phased implementation schedule set out in Section 11.

11. Implementation Roadmap

A five-phase plan, sequenced so that each phase produces a demoable milestone, supporting periodic review with the faculty guide.

Phase	Focus	Key deliverables	Indicative duration
1	Harden the existing base	Add device key generation + message signing (Module A→B); replace fixed thresholds with adaptive per-device baseline (Module C)	2–3 weeks
2	Trust & risk engine	Implement trust-score deltas, bounded risk formula, SAFE/SUSPICIOUS/HIGH/CRITICAL bands (Module D)	2 weeks

Phase	Focus	Key deliverables	Indicative duration
3	Audit ledger + tamper feedback	Build lightweight hash-chain across 2–3 nodes; implement periodic integrity check feeding back into trust score (Module E, Section 8.2)	3–4 weeks
4	Automated response	Quarantine state machine, command blocking at gateway, physical alarm/lock actuation, admin-gated recovery (Module F/G)	2 weeks
5	Dashboard, evaluation & write-up	Security dashboard, attack simulations (brute force, replay, spoofing, log tampering) with measured detection/false-positive rates; draft report	3–4 weeks

Table 1. Suggested phased implementation schedule.

Blockchain platform recommendation: for Phase 3, start with a small custom hash-chain (Python + SQLite, SHA-256, ECDSA signatures) replicated across two or three machines/nodes rather than standing up Hyperledger Fabric. It demonstrates the exact property being claimed (tamper-evidence with cross-node verification) with a fraction of the setup and debugging cost; migrating to Fabric or a similar permissioned platform is a reasonable Phase 6 / future-work item once the mechanism is proven.

12. Indicative Bill of Materials

Item	Role	Approx. qty
ESP32 DevKit (WROOM-32)	Smart home node (Module A)	1
ESP32-S3 DevKit	Security gateway (Module B) — optional; a second WROOM-32 works if S3 is unavailable	1
ESP32 DevKit (WROOM-32)	Physical response node (Module G)	1
DHT11 sensor	Temperature/humidity input	1
Micro servo (SG90-class)	Lock actuator simulation	1
4x4 matrix keypad	Local authentication input	1
OLED display (SSD1306, I2C)	Status/alert display	1
LEDs (red/green) + buzzer	Alarm indicators	1 set
Breadboards, jumper wires, resistors, USB cables	Prototyping	as needed
Laptop/PC running Python 3.x	Detection engine, trust engine, ledger, dashboard	1

Table 2. Hardware/software bill of materials — broadly consistent with the existing prototype, with no exotic or high-cost components required for the core Phase 1–5 build.

13. Risks, Limitations & Open Questions

- The novelty assessment in Sections 8–9 is preliminary and has not been validated against a formal prior-art search; the existing patent landscape may already contain the specific trust↔ledger feedback loop described here.

- A device's private key living in ESP32 flash is protected only as well as the chip's flash-encryption/secure-boot configuration; without those enabled, a physically-present attacker with flashing access could extract it. This should be stated as a known limitation in any write-up, not hidden.
- A permissioned ledger replicated across two or three nodes on the same home network offers weaker Byzantine-fault guarantees than a larger, geographically distributed permissioned network — acceptable for a prototype/demo, but worth flagging explicitly as a scope limitation rather than presenting it as production-grade.
- The ML component (Isolation Forest) needs a reasonable volume of ‘normal’ behavior data to build a meaningful adaptive baseline; early in deployment it will fall back mostly on rules, which should be described honestly in evaluation results.
- Quarantine that blocks a smart lock's commands has real safety implications (a household could be locked in/out); the design should default quarantine to a fail-safe state for the lock specifically, and this should be a documented design decision, not an afterthought.

14. Next Steps

1. Assign module ownership (Section 7) across the team and confirm the phased schedule in Section 11 against team availability.
2. Implement Phases 1–3, prioritizing the trust↔ledger feedback mechanism (Section 8.2), which is the system's principal technical contribution.
3. Conduct evaluation against the attack scenarios listed in Table 1 (brute force, replay, spoofing, log tampering) and record detection and false-positive rates for the final report.
4. Review the completed system with the faculty guide at the end of each phase, and revisit the design decisions in Section 10 if implementation reveals a need to adjust scope.
5. Prior to any public disclosure of the system — including conference or journal submission, a public code repository, or a demonstration recording — consult the institution's intellectual-property office to determine whether patent protection will be pursued.

End of document.